

Bon Bon AI Waiter — Personalization & Insights Blueprint

How YouTube, Instagram Reels, and ChatGPT actually personalize — and exactly how to build the same behaviour into the Bon Bon AI waiter (Gemini 2.5 Flash).

0. The core idea (in one paragraph)

Your instinct is right: a great menu assistant is a **mix of three systems**.

- **Reels/YouTube** = react *fast* from **1–2 signals** and keep a little **exploration** so you don't get boring.
- **ChatGPT** = **infer** what someone really wants from natural language — the stated *and* the unstated.
- **Menu engineering + demand sensing** = roll all those tiny signals up into **owner insights** (what people keep asking for that you don't sell).

The plan below turns each into concrete code you can add to the existing `/api/bonbon` route, the chatbot, and the dashboards. Nothing here needs a trained ML model — it's an LLM (Gemini) doing the inference plus a couple of small database aggregations.

1. How the three systems actually work (condensed + cited)

1a. YouTube — fast retrieval + a watch-time ranker

- Two stages: **candidate generation** (millions → a few hundred) then **ranking** (hundreds → a few shown). (Covington et al., RecSys 2016)
- It **optimizes watch time, not clicks** — because ranking on clicks promotes clickbait people don't finish. (Covington 2016)
- **Cold-start for new users**: lean on cheap context (location, age, time, logged-in) as *priors* so day-one recs are still reasonable. **Cold-start for new items**: an "example age" freshness feature so new uploads aren't buried under old favourites. (Covington 2016)
- Later YouTube added **explicit exploration** via reinforcement learning. (Chen et al., WSDM 2019)

1b. Instagram Reels — react within the session, weigh many signals, act on "not interested"

- The most important predictions for a reel are: will you **watch it through**, **like** it, find it **entertaining**, tap the **audio**. (Instagram official)
- Top signals ≈ **your recent activity**, your history with the poster, info about the reel, info about the poster — recent activity is #1, which is *why 1–2 reels retrain your feed so fast*. (Instagram official)
- **"Not interested"** is a **first-class negative signal** — it suppresses similar content immediately. (Instagram official)

- Explore uses a **retrieval** → **rank** → **rerank funnel**, mixing **real-time** candidate sources (your latest interactions) with **long-term** ones, and a **two-tower** model so a single "like" retrieves nearest-neighbour items instantly. (Meta Engineering, 2023)
- Final score = a **weighted sum of many predicted actions**, with **negative actions ("see less") carrying negative weight**, plus **diversity rules** (don't show two from the same author in a row). (Meta Engineering, 2023)
- Cold-start is handled at scale with **contextual bandits** (explore untried vs exploit known-good). (bandits survey, arXiv 1904.10040)

1c. ChatGPT / conversational recommenders — infer preferences from a short chat

- Conversational recommenders elicit **current, dynamic** preferences through dialogue, not static logs. (CRS survey, arXiv 2004.00646)
- **Don't quiz users on attributes they don't know** ("what fat content?") — ask **usage/goal questions** ("quick treat or something to share?"). (ACM TORS 3629981)
- LLMs can be prompted to **extract implicit (unstated) preferences** and inject them into the recommendation step, which measurably improves results. (MDPI 2025) With **no history, an LLM using only natural-language preferences is competitive with collaborative filtering** — a short chat is enough. (Sanner et al., arXiv 2307.14225)
- **ChatGPT memory** = *saved memories* (explicit facts like "I'm vegetarian") + *reference chat history* (implicit inference), applied by **silently rewriting the request**. Memory can go **stale/contradictory** — so keep it fresh and confirm. (OpenAI Memory FAQ)
- **Watch out for two LLM ranking bugs: position bias** (shuffling the candidate list changes the ranking) and **"lost in the middle"** (models attend to the start/end of a prompt, not the middle). Fixes: rank by shuffling+merging, or pick **one item at a time**. (position bias, lost in the middle)

The one-line synthesis: *retrieve a shortlist → let Gemini infer preferences and re-rank it → keep a slice for exploration → treat "no / not that" as a strong negative → log everything, especially what you couldn't serve.*

2. The Bon Bon blueprint

2a. The taste profile object (the "memory card")

A lightweight JSON the bot fills in from the conversation. Two layers, like ChatGPT: **stable** facts (persist by phone across visits) and **session** context (this visit only). (Schema is our proposal, informed by the six-primary additive taste model and zero-result/unmet-demand analytics — *taste model, zero-result demand*.)

```
{
  "stable": {
    "flavors_liked": ["chocolate", "coffee"],
    "flavors_avoided": ["mango"],
    "taste_axes": { "sweet": 0.8, "nutty": 0.6, "fruity": 0.1, "creamy": 0.7, "bitter_coffee": 0.1 },
    "dietary": { "restrictions": ["eggless"], "allergies": [] },
    "budget_sensitivity": "medium",
    "cultural_cues": ["south_indian"],
```

```

    "kids_in_group": false
  },
  "session": {
    "occasion": ["treat", "sharing"],
    "mood": "something rich and indulgent",
    "temperature_preference": "cold",
    "explicit_asks": ["death by chocolate?"],
    "asked_but_unavailable": [
      { "query": "matcha ice cream", "reason": "not_on_menu" }
    ]
  },
  "meta": { "confidence": 0.7, "source_turns": ["msg_3", "msg_5"], "updated_at": "..."}
}

```

Design rules (from Gemini docs + reliability research): - Use **enums and 0–1 bounded numbers**, not free text, wherever possible — cheaper and more reliable. (Gemini structured output) - Every inferred field carries a **confidence** so weak signals get down-weighted. - `asked_but_unavailable[]` is the **demand-gap ledger** — the single most valuable thing for the owner (see §2f). - Structured output guarantees valid JSON but **not correct values** — validate in code. (Gemini docs)

2b. Extracting the signals with Gemini (you already have the plumbing)

Your `/api/bonbon` route already uses `responseMimeType: "application/json"` + `responseSchema`. Add a **second, cheap Gemini call** (or extend the existing one) whose job is *only* extraction — run it on the user's messages, not every token.

Prompt shape (fits your current route style):

```

SYSTEM: You extract a taste profile from an ice-cream-parlour chat.
- Fill ONLY fields the customer actually supports; leave the rest empty/0.
- Separate STATED facts (allergies, "no mango", budget) from INFERRED ones (mood, occasion) and s
- If they ask for something we don't sell or that's sold out, add it to asked_but_unavailable with
- Never invent preferences.
USER: <the conversation so far> + <the live menu list>

```

Two-layer extraction is the key move: prompt for **stated constraints** *and* **inferred/implicit** signals as separate buckets — that explicit split is what improves downstream recs. (MDPI, Park et al.)

2c. Real-time recommendation strategy (the Reels + ChatGPT mix)

When it's time to suggest dishes, run this funnel — it mirrors YouTube/Reels but is tiny at your catalog size (~110 items):

1. **Retrieve a shortlist (candidate generation).** Filter the menu to what's available *and* compatible with hard constraints (eggless, allergies, budget). From one positive signal, pull "neighbours": same category + shared taste axes (your `catImg` /taste-axis analogue of two-tower nearest-neighbours). (Meta two-tower)
2. **Rank with Gemini, position-bias-safe.** Don't ask "rank these 40." Either (a) ask Gemini to **pick the top 3–5 one at a time**, or (b) rank a **shuffled** list across 2 passes and merge — both cut position bias. Put the shortlist near the **start or end** of the prompt, not buried in the middle. (RISE, lost in middle)

3. **Optimize "will they actually love it," not "will they tap."** The watch-time lesson: bias toward dishes the profile genuinely fits (and that get ordered), not just eye-catching ones. (Covington)
4. **Keep an exploration slot.** Show ~3 safe on-profile picks + **1 novel/adventurous** pick to keep learning and avoid repetition — the bandit remedy for cold-start. (bandits)
5. **Act on negatives instantly.** "Not that / too sweet / no fruit" → write it to `flavors_avoided / taste_axes` and **suppress similar items next turn**, exactly like "Not interested." (Instagram)
6. **Diversify.** Don't suggest three scoops in a row — spread across categories, like Reels' "no two from the same author." (Meta)
7. **Ask usage questions, not attribute quizzes.** One light question ("treat for yourself or sharing with kids?") beats a flavour interrogation. (ACM TORS)

2d. Two layers of memory (this-chat vs across-visits)

- **Session layer** (now): everything above works from the single conversation — that's the Reels "react to 1–2 signals" behaviour. Enough on its own. (Sanner)
- **Stable layer** (later): you already `saveCustomer({phone, last, visits})`. Attach the `stable` block to that record. On a return visit, seed the profile ("Welcome back! Last time you loved the coffee scoop — feeling that again or something new?"). This is ChatGPT's saved-memory pattern. Keep it **fresh** and **confirm** rather than assume, since memory drifts. (OpenAI FAQ)

2e. Guardrails (don't skip these)

- **Confirm inferred preferences** back to the customer before acting ("Sounds like you want something rich and chocolatey — yeah?"). Prevents hallucinated tastes. (critiquing)
- **Validate Gemini's JSON values** in code (allergies especially) — structure is guaranteed, correctness isn't. (Gemini docs)
- **Never override a hard constraint for exploration** (an allergy is not an "explore" opportunity).

2f. Owner insights (rolling the signals up)

Every conversation writes a small signal record. Aggregate them into dashboard reports — these map directly onto established analytics patterns:

1. **Top requested items you don't sell / were sold out** — the demand-gap report. Rank `asked_but_unavailable` by frequency. This is the single highest-value output: latent demand your till never sees. (zero-result demand, demand sensing)
2. **Stock-out demand recovery** — items requested *while marked sold-out*, so their true demand isn't undercounted (sales alone bias demand downward). (demand sensing)
3. **Menu-engineering board** — auto-classify items into **Stars / Plowhorses / Puzzles / Dogs** (popularity × your margin) with a suggested action each. (Toast, Menubly)
4. **"Ordered together" pairings** — market-basket association rules (support/confidence/lift > 1) to build combos and upsell prompts. (market basket, lift)
5. **Taste segments** — cluster customers (sweet-tooth / nutty / coffee / fruity / adventurous) with size + avg spend. (segmentation)
6. **Rising & fading tastes** — week-over-week movers in requested flavours, to time specials.
7. **Unmet-attribute demand** — most-requested attributes you can't satisfy ("vegan", "sugar-free", "matcha"), to guide product development.

8. **Assistant scorecard** — accept-rate of suggestions, plus **coverage** and **popularity-bias** checks so the bot isn't just pushing the same 4 bestsellers. (recsys metrics)
-

3. Suggested build order (mapped to your code)

1. **Signal capture first (cheap, high value)**. Add a Gemini extraction call in `/api/bonbon` that returns the taste profile + `asked_but_unavailable`, and write a `bonbon_signals` Firestore doc per conversation. *You get the demand-gap owner report immediately — before any fancy recommending.*
2. **In-session re-ranking**. Feed the taste profile into the reply prompt; do one-at-a-time top-picks with an exploration slot + negative-signal suppression.
3. **Owner dashboard: demand-gap + menu-engineering boards** (reports #1–3 above).
4. **Cross-visit stable profile** on the existing phone record (welcome-back personalization).
5. **Pairings + segments + scorecard** (reports #4–8) once you have signal volume.

Start with #1 — it's small, needs no new UI in the bot, and gives you the "AI insight" the owner cares about most (what people keep asking for that you don't have).

4. Sources

YouTube / retrieval + ranking - Covington, Adams, Sargin — *Deep Neural Networks for YouTube Recommendations*, RecSys 2016 — <https://cseweb.ucsd.edu/classes/fa17/cse291-b/reading/p191-covington.pdf> - Chen et al. — *Top-K Off-Policy Correction for a REINFORCE Recommender*, WSDM 2019 — <https://research.google/pubs/top-k-off-policy-correction-for-a-reinforce-recommender-system/>

Instagram / Reels / Meta - Instagram — *Shedding More Light on How Instagram Works* — <https://about.instagram.com/blog/announcements/shedding-more-light-on-how-instagram-works> - Meta Engineering — *Scaling the Instagram Explore recommendations system* (2023) — <https://engineering.fb.com/2023/08/09/ml-applications/scaling-instagram-explore-recommendations-system/> - *Practical applications of multi-armed / contextual bandits* (survey) — <https://arxiv.org/pdf/1904.10040>

LLM / conversational recommenders - *A Survey on Conversational Recommender Systems* — <https://arxiv.org/pdf/2004.00646> - *Generating Usage-related Questions for Preference Elicitation* (ACM TORS) — <https://dl.acm.org/doi/full/10.1145/3629981> - *Extracting Implicit User Preferences in CRS Using LLMs* (MDPI 2025) — <https://www.mdpi.com/2227-7390/13/2/221> - Sanner et al. — *LLMs are Competitive Near Cold-start Recommenders* — <https://arxiv.org/abs/2307.14225> - *Evaluating Position Bias in LLM Recommendations (RISE)* — <https://arxiv.org/html/2508.02020v1> - *Lost in the Middle* — <https://arxiv.org/abs/2307.03172> - OpenAI — *ChatGPT Memory FAQ* — <https://help.openai.com/en/articles/8590148-memory-faq>

Gemini structured extraction - Gemini API — *Structured output* — <https://ai.google.dev/gemini-api/docs/structured-output> - Gemini API — *Function calling* — <https://ai.google.dev/gemini-api/docs/function-calling>

Taste / food modeling - Ahn et al. — *Flavor network and the principles of food pairing*, Nature Sci. Reports — <https://www.nature.com/articles/srep00196> - *Preferential Personal Food Model* (six-primary taste space) — <https://arxiv.org/pdf/2008.12855>

Owner analytics / demand / menu engineering - Evidently AI — *Evaluating recommender systems* (metrics) — <https://www.evidentlyai.com/ranking-metrics/evaluating-recommender-systems> - Algo.com — *Demand sensing* (unmet demand, stock-out bias) — <https://www.algo.com/blog/demand-sensing/> - Wizzy — *Zero-result searches as demand signal* — <https://wizzy.ai/blog/zero-result-search-ecommerce/> - Toast — *Menu engineering matrix* — <https://pos.toasttab.com/blog/on-the-line/menu-engineering-matrix> - Analytics Vidhya — *Market basket analysis* — <https://www.analyticsvidhya.com/blog/2021/10/a-comprehensive-guide-on-market-basket-analysis/> - Foodics — *Customer segmentation* — <https://www.foodics.com/customer-segmentation/>

Note: the taste-profile JSON in §2a and the owner-report list in §2f are our design synthesis built on the cited patterns, not verbatim claims from any single source.